

Concepts that you need to know

You only need to know a few concepts to start using Terraform quickly to create the infrastructure you desire. 'Providers' are some of the building blocks in Terraform which abstract different cloud services and back-ends to actually CRUD various resources. Terraform gives you different providers to target different service providers and back-ends, e.g., AWS, Google Cloud, Digital Ocean, Docker and a lot of others. You need to provide different attributes applicable to the targeted service/ back-end like the access/secret keys, regions, endpoints, etc, to enable Terraform to create and manage various cloud/ back-end resources. Different providers offer various resources which correspond to different building blocks, e.g., VMs, storage, networking, managed services, etc. So only a single provider is required to make use of all the resources implemented in Terraform, to create and manage infrastructure for a service or back-end. There are 'provisioners' that correspond to different resources to initialise and configure those resources after their creation. The provisioners mainly do tasks like uploading files, executing remote/local commands/scripts, running configuration management clients, etc.

You need to describe your infrastructure using a simple configuration language in single or multiple files, all with the `.tf` extension. The configuration model of Terraform is declarative, and it mainly merges all the `.tf` files in its working directory at runtime. It resolves the dependencies between various resources by itself to create the correct final dependency graph, to bring up independent resources in parallel. Terraform could use JSON as well for its configuration language, but that works better when Terraform configurations are generated by automated tools. The Terraform format is more human-readable and supports comments, so you could mix and match `.tf` and `.json` configuration files in case some things are human coded and others are tool generated. Terraform also provides the concepts of variables, and functions working on those variables, to store, assign and transform various things at runtime.

The general workflow of Terraform consists of two stages—to plan and apply. The plan stage evaluates the merged (or overridden) configs, and presents a plan before the operator about which resources are going to get created, modified and deleted. So the changes required to create your desired infrastructure are pretty clear at the plan stage itself and there are no surprises at runtime. Once you are satisfied with the plan generated, the apply stage initiates the sequence to create the resources required to build your declared infrastructure. Terraform keeps a record of the created infra in a state file (default, `terraform.tfstate`) and on every further plan-and-apply cycle, it compares the current state of the infra at runtime with the cached state. After

the comparison of states, it only shows or applies the difference required to bring the infrastructure to the desired state as per its configuration. In this way, it creates/maintains the whole infra in an idempotent manner at every apply stage. You could mark various resources manually to get updated in the next apply phase using the *taint* operation. You could also clean up the infra created, partially or fully, with the *destroy* operation.

Working examples and usage

Our first example is to clarify the syntax for various sections in Terraform configuration files. Download the code `example1.tf` from http://opensourceforu.com/article_source_code/sept17/terraform.zip. The code is a template to bring up multiple instances of AWS EC2 VMs with Ubuntu 14.04 LTS and an encrypted EBS data volume, in a specified VPC subnet, etc. The template also does remote provisioning on the instance(s) brought up by transferring a provisioning script and doing some remote execution.

Now, let's dissect this example, line by line, in order to practically explore the Terraform concepts. The lines starting with the keyword *variable* are starting the blocks of input variables to store values. The variable blocks allow the assigning of some initial values used as default or no values at all. In case of no default values, Terraform will prompt for the values at runtime, if these values are not set using the option `-var '<variable>=<value>'`. So, in our example, sensitive data like AWS access/private keys are not being put in the template as it is advisable to supply these at runtime, manually or through the command options or through environment variables. The environment variables should be in the form of `TF_VAR_name` to let Terraform read it. The variables could hold string, list and map types of values, e.g., storing a map of different *amis* and subnets for different AWS regions as demonstrated in our example. The string value is contained in double quotes, lists in square brackets and maps in curly braces. The variables are referenced, and their values are extracted through interpolation at different places using the syntax `${var:<variable name>}`. You could explore everything about Terraform variables on the official variables help page.

It's easy to guess that the block starting with the keyword *provider* is declaring and supplying the arguments for the service/back-end. The different providers take different arguments based upon the service/back-end being used and you could explore those in detail on the official *providers* page. The *resource* keyword contains the main meat in any Terraform configuration. We are using two AWS building blocks in our example: `aws_instance` to bring up instances